EX.NO:12

03/04/2025 **LUCAS KANADE ALGORITHM**

AIM:

To implement the Lucas-Kanade Optical Flow algorithm to track motion in a video and visualize the difference between the first two frames.

Algorithm (Lucas-Kanade Optical Flow with Frame Difference):

1. **Load the Video:**
   - Read the input video using OpenCV (`cv2.VideoCapture`).
   - Resize frames for better efficiency.
2. **Feature Detection:**
   - Use the Shi-Tomasi method to detect good feature points in the first frame.
3. **Compute Frame Difference:**
   - Convert the first two frames to grayscale.
   - Compute absolute difference between them and display it.
4. **Track Motion using Lucas-Kanade Method:**
   - Calculate optical flow between consecutive frames using `cv2.calcOpticalFlowPyrLK()`.
   - Draw motion vectors to visualize movement.
5. **Store and Display Results:**
   - Store the processed frames in a video file.
   - Display output directly in Jupyter Notebook.

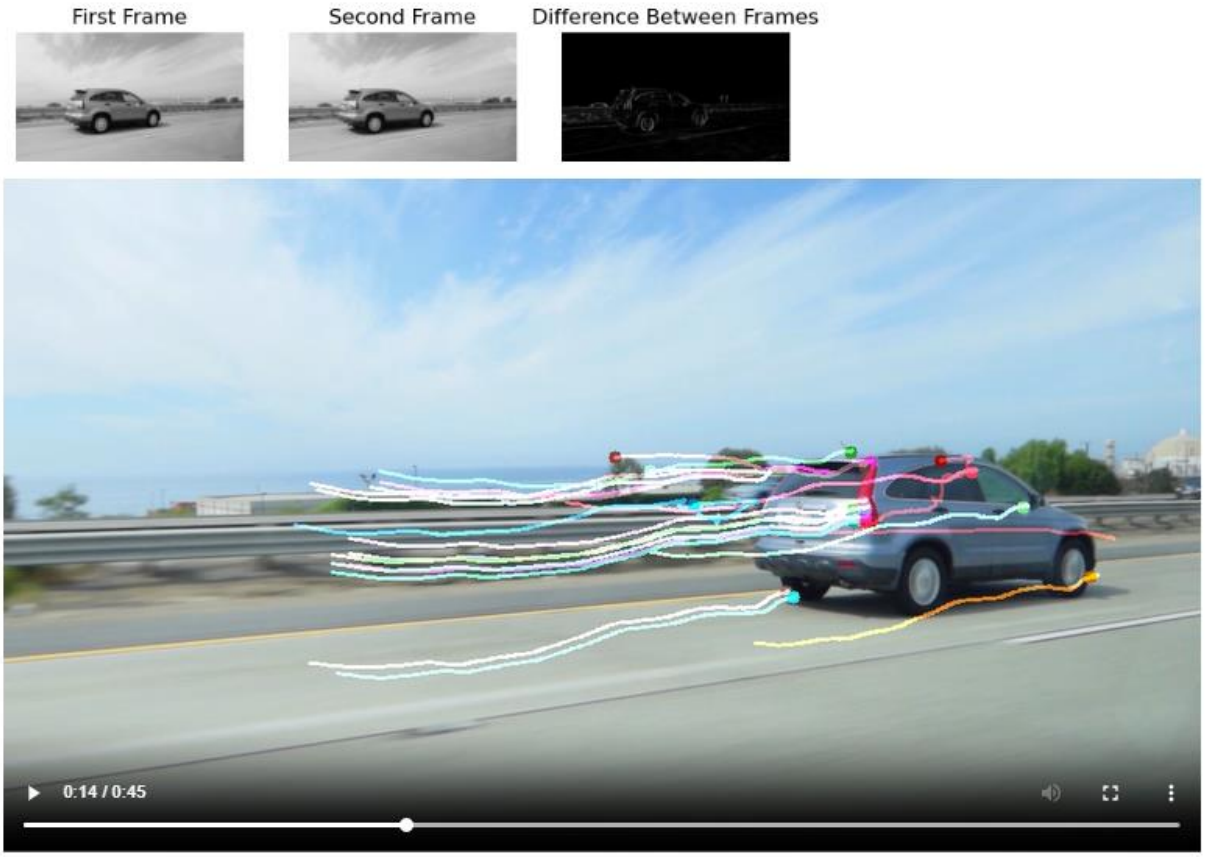
CODE SNIPPET:

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from IPython.display import display, Video

def lucas_kanade_optical_flow(video_path, output_path="output.mp4"):

    cap = cv2.VideoCapture(video_path)

    width = int(cap.get(3) // 2)

    height = int(cap.get(4) // 2)

    fourcc = cv2.VideoWriter_fourcc(*'mp4v')

    out = cv2.VideoWriter(output_path, fourcc, 20.0, (width, height))
```

```
feature_params = dict(maxCorners=100, qualityLevel=0.3, minDistance=7, blockSize=7)
lk_params = dict(winSize=(15, 15), maxLevel=2,
                 criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03))
color = np.random.randint(0, 255, (100, 3))
ret, old_frame = cap.read()
if not ret:
    print("Error: Cannot read video.")
    return
old_frame = cv2.resize(old_frame, (width, height))
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)
mask = np.zeros_like(old_frame)
ret, frame = cap.read()
if not ret:
    print("Error: Not enough frames.")
    return
frame = cv2.resize(frame, (width, height))
frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
frame_diff = cv2.absdiff(old_gray, frame_gray)
plt.figure(figsize=(8, 4))
plt.subplot(1, 3, 1)
plt.imshow(old_gray, cmap='gray')
plt.title("First Frame")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(frame_gray, cmap='gray')
plt.title("Second Frame")
plt.axis("off")
```

```
    plt.subplot(1, 3, 3)
    plt.imshow(frame_diff, cmap='gray')
    plt.title("Difference Between Frames")
    plt.axis("off")
    plt.show()
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame = cv2.resize(frame, (width, height))
        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray, frame_gray, p0, None, **lk_params)
        if p1 is not None:
            good_new = p1[st == 1]
            good_old = p0[st == 1]
            for i, (new, old) in enumerate(zip(good_new, good_old)):
                a, b = new.ravel()
                c, d = old.ravel()
                mask = cv2.line(mask, (int(a), int(b)), (int(c), int(d)), color[i].tolist(), 2)
                frame = cv2.circle(frame, (int(a), int(b)), 5, color[i].tolist(), -1)
        final_output = cv2.add(frame, mask)
        out.write(final_output)
        old_gray = frame_gray.copy()
        p0 = good_new.reshape(-1, 1, 2) if p1 is not None else p0
    cap.release()
    out.release()
    display(Video(output_path, embed=True))
lucas_kanade_optical_flow(r"C:\Users\student\Downloads\car.mp4")
```

OUTPUT:



RESULT:

Thus the program to implement the Lucas-Kanade Optical Flow algorithm to track motion in a video and visualize the difference between the first two frames was executed successfully.